

Name : Vaibhav Gangadhar Alagi.

Project Name : E-Commerce Order Management & Analytics System.

1. Why did you choose this project?

Answer:

I built this project to understand how real business transactional data is stored and how SQL is used to generate insights such as revenue trends, customer behavior, and product performance.

2. What is your project about?

Answer:

This project is a relational database system designed using MySQL to manage customers, orders, products, and payments, and to perform analytical queries for business reporting.

3. Which tools and technologies did you use?

Answer:

I used **MySQL** and advanced SQL features such as joins, subqueries, CTEs, and window functions.

4. How many tables did you design and why?

Answer:

I designed five tables:

customers, products, orders, order_items, and payments.

This structure avoids data duplication and follows normalization up to 3NF.

5. Explain your tables briefly.

Answer:

- customers → stores customer details
- products → stores product information
- orders → stores order level information
- order_items → stores which products belong to each order
- payments → stores payment details for each order

6. How did you design the database?

Answer:

First, I identified entities such as customer, product, order, and payment.

Then I created normalized tables with primary keys and foreign keys.

After that, I added constraints like NOT NULL, UNIQUE, and CHECK to ensure data integrity.

7. How did you complete this project?

Answer:

1. Created database and tables
2. Defined primary and foreign keys

3. Added constraints and indexes
4. Inserted realistic sample data
5. Wrote analytical SQL queries for business reporting

8. What type of SQL concepts did you use?

Answer:

I used joins, subqueries, aggregations, CTEs, and window functions.

9 . Where did you use window functions?

Answer:

I used RANK and DENSE_RANK to rank customers based on total orders and spending, and to find top products within each category.

10. How is this project useful for a business?

Answer:

It helps businesses identify top customers, popular products, revenue trends, and preferred payment modes, which supports better decision making.

11. Did you consider performance?

Answer:

Yes, I created indexes on frequently joined columns such as customer_id and product_id to improve query performance.

12. What was the main challenge in this project?

Answer:

Designing correct relationships and writing accurate analytical queries using multiple joins and window functions.

13. What did you learn from this project?

Answer:

I gained strong understanding of relational database design, normalization, SQL analytics, and real-world business reporting using SQL.

14. Why did you use MySQL?

Answer:

MySQL is widely used in industry and is very suitable for learning relational database fundamentals and analytics queries.

15. What will you improve in this project?

Answer:

I plan to add inventory management, user roles, and connect this database to a reporting or BI tool.

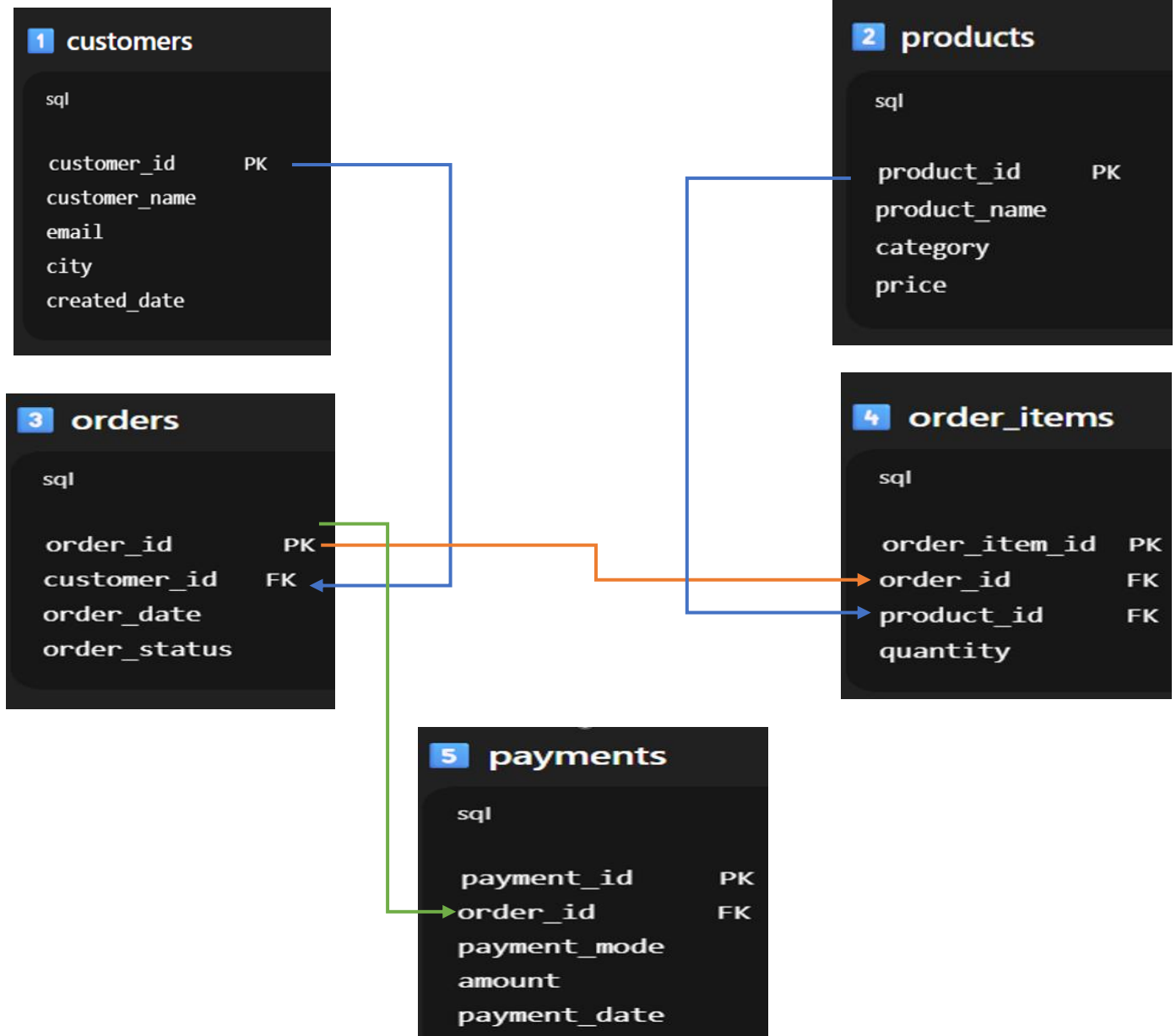
Total Create table 5

Table Name

1. customers
2. products
3. orders
4. order_items
5. payments

ER Diagram :

```
Customers ---< Orders ---< Order_Items >--- Products
              |
              |
              +--- Payments
```



```
CREATE TABLE customers (  
    customer_id INT PRIMARY KEY,  
    customer_name VARCHAR(100) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    city VARCHAR(50),  
    created_date DATE DEFAULT (CURDATE())  
);
```

```
CREATE TABLE orders (  
    order_id INT PRIMARY KEY,  
    customer_id INT NOT NULL,  
    order_date DATE NOT NULL,  
    order_status VARCHAR(20),  
    CONSTRAINT fk_orders_customer  
        FOREIGN KEY (customer_id)  
        REFERENCES customers(customer_id)  
);
```

```
CREATE TABLE payments (  
    payment_id INT PRIMARY KEY,  
    order_id INT UNIQUE NOT NULL,  
    payment_mode VARCHAR(20),  
    amount DECIMAL(10,2) CHECK (amount >= 0),  
    payment_date DATE,  
    CONSTRAINT fk_payments_order  
        FOREIGN KEY (order_id)  
        REFERENCES orders(order_id)  
);
```

```
CREATE TABLE products (  
    product_id INT PRIMARY KEY,  
    product_name VARCHAR(100) NOT NULL,  
    category VARCHAR(50),  
    price DECIMAL(10,2) CHECK (price >= 0)  
);
```

```
CREATE TABLE order_items (  
    order_item_id INT PRIMARY KEY,  
    order_id INT NOT NULL,  
    product_id INT NOT NULL,  
    quantity INT CHECK (quantity > 0),  
    CONSTRAINT fk_order_items_order  
        FOREIGN KEY (order_id)  
        REFERENCES orders(order_id),  
    CONSTRAINT fk_order_items_product  
        FOREIGN KEY (product_id)  
        REFERENCES products(product_id)  
);
```

Because customer_id and product_id are heavily used in JOIN and filtering operations, so indexing them reduces table scan and speeds up analytics queries.

These are non-unique indexes created to improve query performance on frequently joined columns.

```
CREATE INDEX idx_orders_customer
```

```
ON orders(customer_id);
```

```
CREATE INDEX idx_order_items_product
```

```
ON order_items(product_id);
```

Insert all record inside the Table :

INSERT INTO customers (customer_id, customer_name, email, city) VALUES

(1, 'Amit Sharma', 'amit@gmail.com', 'Pune'),
(2, 'Neha Patil', 'neha@gmail.com', 'Mumbai'),
(3, 'Rahul Verma', 'rahul@gmail.com', 'Delhi'),
(4, 'Sneha Kulkarni', 'sneha@gmail.com', 'Pune'),
(5, 'Rohit Mehta', 'rohit@gmail.com', 'Ahmedabad'),
(6, 'Pooja Singh', 'pooja@gmail.com', 'Bangalore'),
(7, 'Karan Malhotra', 'karan@gmail.com', 'Delhi'),
(8, 'Anjali Deshmukh', 'anjali@gmail.com', 'Nagpur'),
(9, 'Suresh Iyer', 'suresh@gmail.com', 'Chennai'),
(10, 'Nikita Joshi', 'nikita@gmail.com', 'Mumbai'),
(11, 'Vikas Rao', 'vikas@gmail.com', 'Hyderabad'),
(12, 'Ritu Jain', 'ritu@gmail.com', 'Indore'),
(13, 'Manish Gupta', 'manish@gmail.com', 'Jaipur'),
(14, 'Kavita Nair', 'kavita@gmail.com', 'Kochi'),
(15, 'Arjun Kapoor', 'arjun@gmail.com', 'Delhi'),
(16, 'Priya Shah', 'priya@gmail.com', 'Surat'),
(17, 'Deepak Yadav', 'deepak@gmail.com', 'Noida'),
(18, 'Swati More', 'swati@gmail.com', 'Pune'),
(19, 'Nilesh Patil', 'nilesh@gmail.com', 'Kolhapur'),
(20, 'Ramesh Pawar', 'ramesh@gmail.com', 'Satara');

INSERT INTO products (product_id, product_name, category, price) VALUES

(101, 'Laptop', 'Electronics', 55000),
(102, 'Mobile Phone', 'Electronics', 25000),
(103, 'Headphones', 'Electronics', 2000),
(104, 'Smart Watch', 'Electronics', 5000),

(105, 'Bluetooth Speaker', 'Electronics', 3500),
(106, 'T-Shirt', 'Clothing', 800),
(107, 'Jeans', 'Clothing', 1800),
(108, 'Shoes', 'Clothing', 3000),
(109, 'Jacket', 'Clothing', 4500),
(110, 'Sofa Cover', 'Home', 1200),
(111, 'Dining Set', 'Home', 25000),
(112, 'Mixer Grinder', 'Home', 4000),
(113, 'Rice Cooker', 'Home', 3000),
(114, 'Notebook', 'Stationery', 100),
(115, 'Pen Pack', 'Stationery', 150),
(116, 'Backpack', 'Accessories', 2000),
(117, 'Sunglasses', 'Accessories', 1500),
(118, 'Wallet', 'Accessories', 1200),
(119, 'Water Bottle', 'Accessories', 600),
(120, 'Office Chair', 'Furniture', 8000);

INSERT INTO orders (order_id, customer_id, order_date, order_status) VALUES

(1001, 1, '2024-01-05', 'DELIVERED'),
(1002, 2, '2024-01-06', 'DELIVERED'),
(1003, 3, '2024-01-08', 'CANCELLED'),
(1004, 4, '2024-01-10', 'DELIVERED'),
(1005, 5, '2024-01-12', 'DELIVERED'),
(1006, 6, '2024-01-15', 'DELIVERED'),
(1007, 7, '2024-01-18', 'DELIVERED'),
(1008, 8, '2024-01-20', 'DELIVERED'),
(1009, 9, '2024-01-22', 'DELIVERED'),
(1010, 10, '2024-01-25', 'DELIVERED'),
(1011, 1, '2024-02-01', 'DELIVERED'),

(1012, 2, '2024-02-03', 'DELIVERED'),
(1013, 3, '2024-02-05', 'DELIVERED'),
(1014, 4, '2024-02-07', 'DELIVERED'),
(1015, 5, '2024-02-10', 'RETURNED'),
(1016, 6, '2024-02-12', 'DELIVERED'),
(1017, 7, '2024-02-15', 'DELIVERED'),
(1018, 8, '2024-02-18', 'DELIVERED'),
(1019, 9, '2024-02-20', 'DELIVERED'),
(1020, 10, '2024-02-22', 'DELIVERED'),
(1021, 11, '2024-03-01', 'DELIVERED'),
(1022, 12, '2024-03-03', 'DELIVERED'),
(1023, 13, '2024-03-05', 'DELIVERED'),
(1024, 14, '2024-03-07', 'DELIVERED'),
(1025, 15, '2024-03-10', 'DELIVERED'),
(1026, 16, '2024-03-12', 'DELIVERED'),
(1027, 17, '2024-03-15', 'DELIVERED'),
(1028, 18, '2024-03-18', 'DELIVERED'),
(1029, 19, '2024-03-20', 'DELIVERED'),
(1030, 20, '2024-03-22', 'DELIVERED');

INSERT INTO order_items (order_item_id, order_id, product_id, quantity) VALUES

(1,1001,101,1),(2,1001,103,2),
(3,1002,102,1),(4,1002,105,1),
(5,1004,106,3),(6,1004,107,2),
(7,1005,108,1),(8,1006,109,1),
(9,1007,110,2),(10,1008,111,1),
(11,1009,112,1),(12,1010,113,1),
(13,1011,101,1),(14,1011,104,1),
(15,1012,102,1),(16,1013,103,3),

(17,1014,105,2),(18,1015,106,2),
(19,1016,107,1),(20,1017,108,2),
(21,1018,109,1),(22,1019,110,2),
(23,1020,111,1),(24,1021,112,1),
(25,1022,113,1),(26,1023,114,5),
(27,1024,115,4),(28,1025,116,1),
(29,1026,117,2),(30,1027,118,1),
(31,1028,119,3),(32,1029,120,1),
(33,1030,101,1),(34,1030,103,2),
(35,1026,105,1),(36,1027,106,2),
(37,1028,107,1),(38,1029,108,1),
(39,1021,109,1),(40,1022,110,2);

INSERT INTO payments (payment_id, order_id, payment_mode, amount, payment_date) VALUES

(201,1001,'UPI',59000,'2024-01-05'),
(202,1002,'CARD',28500,'2024-01-06'),
(203,1004,'UPI',4200,'2024-01-10'),
(204,1005,'CARD',3000,'2024-01-12'),
(205,1006,'UPI',4500,'2024-01-15'),
(206,1007,'NETBANKING',2400,'2024-01-18'),
(207,1008,'UPI',25000,'2024-01-20'),
(208,1009,'CARD',4000,'2024-01-22'),
(209,1010,'UPI',3000,'2024-01-25'),
(210,1011,'CARD',60000,'2024-02-01'),
(211,1012,'UPI',25000,'2024-02-03'),
(212,1013,'CARD',6000,'2024-02-05'),
(213,1014,'UPI',7000,'2024-02-07'),
(214,1015,'CARD',1600,'2024-02-10'),

(215,1016,'UPI',1800,'2024-02-12'),
(216,1017,'UPI',6000,'2024-02-15'),
(217,1018,'CARD',4500,'2024-02-18'),
(218,1019,'UPI',2400,'2024-02-20'),
(219,1020,'NETBANKING',25000,'2024-02-22'),
(220,1021,'UPI',4000,'2024-03-01'),
(221,1022,'CARD',3000,'2024-03-03'),
(222,1023,'UPI',500,'2024-03-05'),
(223,1024,'UPI',600,'2024-03-07'),
(224,1025,'CARD',2000,'2024-03-10'),
(225,1026,'UPI',3000,'2024-03-12'),
(226,1027,'UPI',1500,'2024-03-15'),
(227,1028,'CARD',1800,'2024-03-18'),
(228,1029,'UPI',8000,'2024-03-20'),
(229,1030,'CARD',59000,'2024-03-22');

ADVANCED SQL ANALYTICS QUERIES

E-Commerce Order Management System (MySQL)

1 TOTAL REVENUE --

```
SELECT SUM(amount) AS total_revenue  
FROM payments;
```

2 REVENUE BY MONTH --

```
SELECT  
    DATE_FORMAT(payment_date, '%Y-%m') AS month,  
    SUM(amount) AS monthly_revenue  
FROM payments  
GROUP BY month  
ORDER BY month;
```

3 TOP 5 CUSTOMERS BY SPENDING (WINDOW FUNCTION)

```
SELECT  
    customer_name,  
    total_spent,  
    RANK() OVER (ORDER BY total_spent DESC) AS rank_no  
FROM (  
    SELECT  
        c.customer_name,  
        SUM(p.amount) AS total_spent  
    FROM customers c  
    JOIN orders o ON c.customer_id = o.customer_id  
    JOIN payments p ON o.order_id = p.order_id  
    GROUP BY c.customer_name  
    ) t  
LIMIT 5;
```

4 CUSTOMER LIFETIME VALUE (CLV)

```
SELECT  
  
    c.customer_name,  
  
    COUNT(o.order_id) AS total_orders,  
  
    SUM(p.amount) AS lifetime_value  
  
FROM customers c  
  
JOIN orders o ON c.customer_id = o.customer_id  
  
JOIN payments p ON o.order_id = p.order_id  
  
GROUP BY c.customer_name;
```

5 REPEAT CUSTOMERS

```
SELECT  
  
    customer_id,  
  
    COUNT(order_id) AS order_count  
  
FROM orders  
  
GROUP BY customer_id  
  
HAVING COUNT(order_id) > 1;
```

6 CUSTOMERS WITH NO ORDERS (ANTI JOIN)

```
SELECT c.*  
  
FROM customers c  
  
LEFT JOIN orders o ON c.customer_id = o.customer_id  
  
WHERE o.order_id IS NULL;
```

7 BEST SELLING PRODUCTS (QUANTITY)

```
SELECT  
  
    p.product_name,  
  
    SUM(oi.quantity) AS total_quantity  
  
FROM products p  
  
JOIN order_items oi ON p.product_id = oi.product_id
```

GROUP BY p.product_name

ORDER BY total_quantity DESC;

8 PRODUCT REVENUE CONTRIBUTION

SELECT

p.product_name,

SUM(oi.quantity * p.price) AS product_revenue

FROM products p

JOIN order_items oi ON p.product_id = oi.product_id

GROUP BY p.product_name

ORDER BY product_revenue DESC;

9 MOST POPULAR PAYMENT MODE

SELECT

payment_mode,

COUNT(*) AS usage_count

FROM payments

GROUP BY payment_mode

ORDER BY usage_count DESC;

10 AVERAGE ORDER VALUE (AOV)

SELECT

AVG(order_total) AS avg_order_value

FROM (

SELECT

order_id,

SUM(amount) AS order_total

FROM payments

GROUP BY order_id

) t;

1 1 MONTH-OVER-MONTH SALES GROWTH

WITH monthly_sales AS (

```

SELECT
    DATE_FORMAT(payment_date, '%Y-%m') AS month,
    SUM(amount) AS revenue
FROM payments
GROUP BY month
)
SELECT
    month,
    revenue,
    revenue - LAG(revenue) OVER (ORDER BY month) AS growth
FROM monthly_sales;

```

===== **VERY IMPORTANT – interview favorite**=====

1 2 CUSTOMER RANKING BY ORDERS

```

SELECT
    customer_name,
    total_orders,
    DENSE_RANK() OVER (ORDER BY total_orders DESC) AS rank_no
FROM (
    SELECT
        c.customer_name,
        COUNT(o.order_id) AS total_orders
    FROM customers c
    JOIN orders o ON c.customer_id = o.customer_id
    GROUP BY c.customer_name
);

```

1 3 TOP CATEGORY BY SALES

```
SELECT  
  
    category,  
  
    SUM(oi.quantity * p.price) AS category_sales  
  
FROM products p  
  
JOIN order_items oi ON p.product_id = oi.product_id  
  
GROUP BY category  
  
ORDER BY category_sales DESC;
```

1 4 CANCELLED / RETURNED ORDERS COUNT

```
SELECT  
  
    order_status,  
  
    COUNT(*) AS count_orders  
  
FROM orders  
  
WHERE order_status IN ('CANCELLED', 'RETURNED')  
  
GROUP BY order_status;
```

1 5 HIGH-VALUE ORDERS (> AVG ORDER VALUE)

```
SELECT  
  
    order_id,  
  
    SUM(amount) AS order_amount  
  
FROM payments  
  
GROUP BY order_id  
  
HAVING SUM(amount) > (  
  
    SELECT AVG(amount) FROM payments  
  
);
```

1 6 CUSTOMER FIRST & LAST PURCHASE DATE

```
SELECT  
  
    c.customer_name,
```

```
MIN(o.order_date) AS first_order,  
MAX(o.order_date) AS last_order  
FROM customers c  
JOIN orders o ON c.customer_id = o.customer_id  
GROUP BY c.customer_name;
```

1 7 TOP 3 PRODUCTS PER CATEGORY (WINDOW FUNCTION)

```
SELECT *  
FROM (  
    SELECT  
        p.category,  
        p.product_name,  
        SUM(oi.quantity) AS qty,  
        RANK() OVER (  
            PARTITION BY p.category  
            ORDER BY SUM(oi.quantity) DESC  
        ) AS rank_no  
    FROM products p  
    JOIN order_items oi ON p.product_id = oi.product_id  
    GROUP BY p.category, p.product_name)  
WHERE rank_no <= 3;
```

1 8 DAILY SALES TREND

```
SELECT  
    payment_date,  
    SUM(amount) AS daily_sales  
FROM payments  
GROUP BY payment_date  
ORDER BY payment_date;
```